Objetivo: Aprender las técnicas básicas de depuración de programas en C para revisar de manera precisa el flujo de ejecución de un programa y el valor de las variables; en su caso, corregir posibles errores.

Antecedentes:

El compilador antes de generar el programa objeto debe analizar el programa fuente (almacenado en un archivo de texto) y comprobar que cumple las reglas de sintaxis del lenguaje de programación. Si cumple la sintaxis genera el correspondiente programa objeto, sino, visualizará un mensaje de error y su posible causa para que procedamos a la depuración del programa. Depurar un programa consiste en detectar y corregir los errores que se producen en él, ya sean en tiempo de compilación o en tiempo de ejecución. Los errores en tiempo de compilación son los que se producen antes de la ejecución del programa, durante el proceso de compilación del programa. Los errores en tiempo de ejecución son los que se producen durante la ejecución del programa. Este tipo de errores, son los más difíciles de encontrar, porque no son detectados por el compilador, ya que no son errores consecuencia de violar alguna regla sintáctica. Es decir, aunque al compilar un programa no de errores, eso no significa que el programa funcione adecuadamente o que no vaya a producir algún fallo durante la ejecución del mismo. Por ejemplo:

- *Un programa puede producir resultados erróneos, porque nos hayamos equivocado (errores lógicos) al programar el algoritmo (por ejemplo, donde teníamos que sumar, hemos programado una multiplicación).*

- *Un programa puede interrumpirse bruscamente, por ejemplo si tenemos que hacer una división y el divisor es cero, etc.*

Para ayudarnos a encontrar los errores lógicos y demás errores producidos en tiempo de ejecución, podemos hacer uso de los depuradores.

Es el proceso de identificar y corregir errores de programación. En inglés se conoce como debugging, ya que se asemeja a la eliminación de bichos. Si bien existen técnicas para la revisión sistemática del código fuente y se cuenta con medios computacionales para la detección de errores (depuradores), sigue siendo en buena medida una actividad manual, que desafía la paciencia, la imaginación y la intuición del programador. Muchas veces se requiere incluir en el código fuente instrucciones auxiliares que permitan el seguimiento de la ejecución del programa, presentado los valores de variables y direcciones de memoria y ralentizando la salida de datos.

Los compiladores emiten mensajes de error o de advertencia durante las fases de compilación, de enlace o de ejecución de un programa.

Los mensajes de error producidos los podemos agrupar en tres bloques:

- ***<u>Errores de sintaxis</u>**: Los errores de sintaxis son los más fáciles de detectar en el código fuente de un programa. Un **error de sintaxis** se produce al escribir, incorrectamente, alguna parte del código.*

```
include<stdio.h>
imt main()
{
  inta, b;
   a = 4;
   b = a * 6;
   printf( "%d", b )
   retunr 0;
}
```

Existen cuatro errores de sintaxis:

- Se ha escrito include en vez de #include.
- Se ha escrito imt en vez de int.
- Falta un punto y coma (;) después de la llamada a printf.
- Se ha escrito retunr en vez de return.

- ***Errores de ejecución****: Una vez corregidos los errores de sintaxis en el código fuente de un programa, se debe revisar que no se puede producir ningún error en la ejecución del mismo. Un* ***error de ejecución*** *se produce cuando la computadora no puede ejecutar alguna instrucción de forma correcta.*

```
#include <stdio.h>
int main()
{
  int a;
  float b;
  a = 0;
  b = 6.4 / a;
  printf( "%f", b );
  return 0;
}
```

Se produce un error de ejecución, ya que, al intentar evaluar la expresión:

6.4 / a

a vale 0, y no se puede dividir entre cero.

- ***Los errores de lógica*** *son los más difíciles de detectar. Cuando un programa no tiene errores de sintaxis ni de ejecución, pero, aún así, no funciona bien, esto es debido a la existencia de algún error lógico. De manera que,* ***un error de lógica*** *se produce cuando los resultados obtenidos no son los esperados.*

```
#include <stdio.h>
int main()
{
  float base, altura;
  base = 6.3;
  altura = 4.;
  printf( "El area es: %f", base * altura / 3 );
  return 0;
}
```

Su salida por pantalla es:

```
El área es: 8.4
```

No existen errores de sintaxis ni de ejecución, sin embargo, la salida esperada es:

```
El área es: 12.6
```

Por consiguiente, existe algún error lógico que hay que corregir. El error está en la expresión:
*base * altura / 3*
que, aunque no es incorrecta, no sirve para calcular el área de un triángulo. En su lugar se debe escribir:
*base * altura / 2*

- ***Advertencias (warnings):*** *No impiden la compilación. Indican condiciones que son sospechosas, pero son legítimas como parte del lenguaje.*

Así pues los pasos a seguir para la elaboración y ejecución de un programa los podemos resumir de la siguiente manera:

1 º. Comenzamos escribiendo el código fuente.

2 º. Compilamos el fichero fuente y se comprueban los mensajes de error.

3 º. Volvemos al editor y eliminamos los errores de sintaxis.

4 º. Cuando el compilador tiene éxito, el enlazador construirá el archivo ejecutable.

5 º. Ya podemos ejecutar el archivo ejecutable que se obtiene al enlazar.

6 º. Si encontramos un error, podemos activar el depurador para trazar el programa y ejecutar sentencia a sentencia.

7 º. Una vez que hayamos encontrado la causa del error, volveremos al editor y lo corregimos.

8 º. El proceso de compilar, enlazar y ejecutar el programa lo repetiremos hasta que no se produzcan errores.

9 º. Una vez tengamos el archivo ejecutable, será el sistema operativo el encargado de colocar el programa en la memoria central y ejecutarlo.

Desarrollo:

Realizar los siguientes ejercicios. Hacer su depuración considerando lo siguiente:

- *Realizar el análisis de las variables (realizar impresiones de pantalla) observando su comportamiento.*
- *Identificar cual es el funcionamiento de cada programa.*
- *Señalar el tipo o los tipos de errores que tiene cada programa.*
- *Corregir mediante el depurador de DevC los errores y mostrar el correcto funcionamiento de cada programa.*

Programa 1

```
1   #include <stdio.h>
2   void main()
3   {
4
5
6       int N,CONT, AS;
7       AS=0;
8       CONT=1;
9       printf ("dame numero");
10      scanf ("%f",&N);
11      while(CONT<=N)
12      {
13          AS=(AS+CONT);
14          CONT=(CONT+2);
15      }
16      printf("\nEL RESULTADO ES %i\n",AS);
17
18  }
```

Programa 2

```
1   #include <stdio.h>
2   main()
3   {int i,j;
4
5   for (i=1;i<10;i++);
6   {
7
8   printf("\ntabla del %i\n",i);
9
10      for(j=1;j==10;j++)
11      {
12
13      printf("%i X %i =%i\n",i,j,i*j);
14
15      }
16
17  }
18  }
19
20
```

Programa 3

```
1   #include <stdio.h>
2   #include <math.h>
3   main()
4   {int K,X,AP,N;
5   float AS;
6   printf ("el termino generico de la serie es: X^K/K!");
7   printf ("\nN=");
8   scanf("%d",N);
9   printf ("X=");
10  scanf("%d",X);
11  K=0;
12  AP=0;
13  AS=0;
14  while(K<=N)
15  { AS=AS+pow (X,K)/AP;
16      K=K+1;
17      AP=AP*K;
18
19  }
20  printf("SUM=%i",AS);
21  }
```